emSigner Gateway Integration

Integration document

Document Version 2.0 | Sept 2024

eMudhra | www.emudhra.com

Contents

1. Abbreviation

The below abbreviations are used in the document.

ESGS	emSigner Signer gateway service

2. Overview

emSigner signer gateway is the secured automated signing solution with different signature options (dSign/ eSignature). By using emSigner signer gateway, you can get signatures easily i.e. by sending https post request. Request contains Authentication token to identify the user, document to be signed like PDF, DATA and XML, signature methods, success URL to be sent etc.

On successful post, host application will redirect you to emSigner signer gateway, where you can sign the document with a preview of document (based on the request). Finally signed file or data will be sent back to the response URL in the base64 string format. User data that is sent is nowhere saved on the server. This service can be integrated with all the applications. We also support eMandate XML signing.

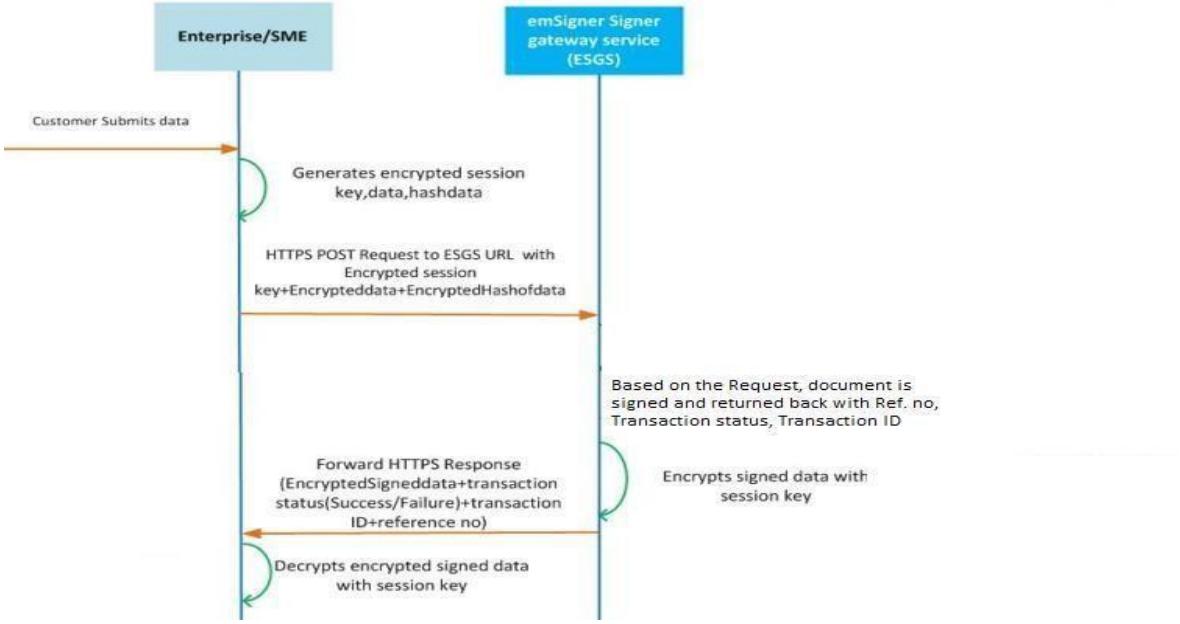
2.1 Purpose

The purpose of the document is to provide users understand the emSigner signer gateway functionality and securely send the document signing request without any integration.

2.2 Target Audience

- Developers
- Enterprise/SME

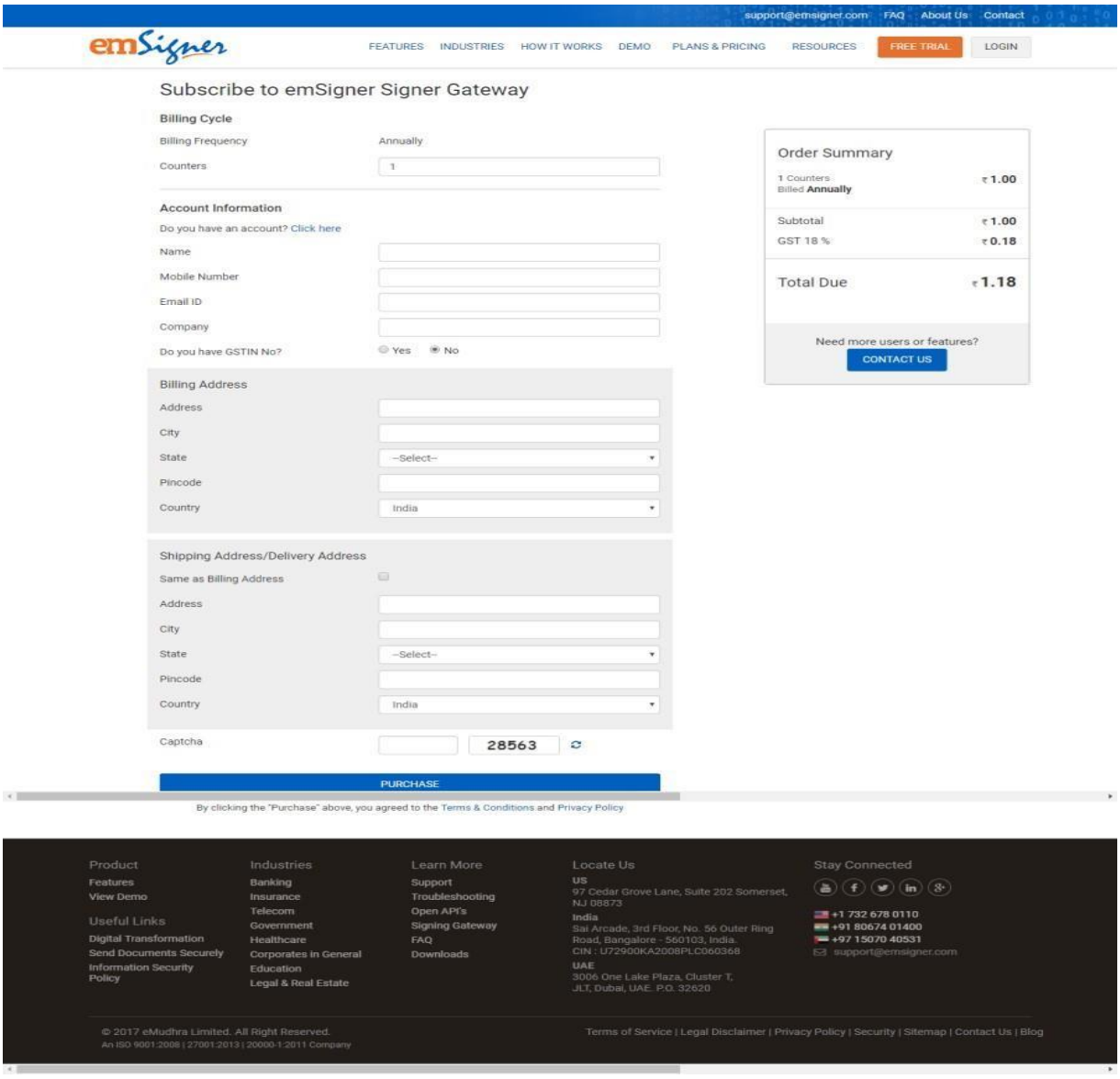
2.3 ESGS Data flow diagram



3. Subscription

3.1 How to create subscription

- To create emSigner signer gateway account, follow below steps:
- Enter www.emsigner.com in the browser.
- In the emSigner top menu, Click on Plans & PricingSigner gateway tab. You will be redirected to a price page, Click on Buy Now button. You are redirected to below page.
- Enter number of counters required for signer gateway and enter the required details. If you are registered user, click on click here link and login with emSigner credentials.
- Click on Purchase button, now you will be redirected to payment gateway page.
- Complete payment. On successful payment you would receive mail with emSigner credentials of one year subscription. Please note that you would be asked to change password for the first login.
- If subscription expires automatically counters will be expired.


support@emsigner.com
FAQ
About Us
Contact
emSigner
FEATURES
INDUSTRIES
HOW IT WORKS
DEMO
PLANS & PRICING
RESOURCES
FREE TRIAL
LOGIN
Subscribe to emSigner Signer Gateway
Billing Cycle
Billing Frequency
Annually
Counters
1
Account Information
Do you have an account? Click here
Name
Mobile Number
Email ID
Company
Do you have GSTIN No?
Yes
No
Billing Address
Address
City
State
--Select--
Pincode
Country
India
Shipping Address/Delivery Address
Same as Billing Address
Address
City
State
--Select--
Pincode
Country
India
Captcha
28563
PURCHASE
By clicking the "Purchase" above, you agreed to the Terms & Conditions and Privacy Policy
Order Summary
1 Counters
Billed Annually
₹ 1.00
Subtotal
₹ 1.00
GST 18 %
₹ 0.18
Total Due
₹ 1.18
Need more users or features?
CONTACT US
Product
Features
View Demo
Useful Links
Digital Transformation
Send Documents Securely
Information Security Policy
Industries
Banking
Insurance
Telecom
Government
Healthcare
Corporates in General
Education
Legal & Real Estate
Learn More
Support
Troubleshooting
Open API's
Signing Gateway
FAQ
Downloads
Locate Us
US
97 Cedar Grove Lane, Suite 202 Somerset, NJ 08873
India
Sai Arcade, 3rd Floor, No. 56 Outer Ring Road, Bangalore - 560103, India.
CIN : U72900KA2008PLC060368
UAE
3006 One Lake Plaza, Cluster T, JLT, Dubai, UAE. P.O. 32620
Stay Connected
+1 732 678 0110
+91 80674 01400
+97 15070 40531
support@emsigner.com
© 2017 eMudhra Limited. All Right Reserved.
An ISO 9001:2008 | 27001:2013 | 20000-1:2011 Company
Terms of Service | Legal Disclaimer | Privacy Policy | Security | Sitemap | Contact Us | Blog

3.2 How to renew subscription

To renew emSigner account, follow below steps:

- Login to your emSigner account with valid credentials.
- In the emSigner dashboard, Click on subscription tab in the left navigator. You will be redirected to a subscription page, Click on Renew button.
- Enter the required details.
- Click on Purchase button, now you will be redirected to payment gateway page.
- Complete payment. On successful payment you would receive mail with successful subscription renewal. Please note that you can add counters only after complete usage of counters with active subscription.

3.3 How to add counters

To add counters in signer gateway emSigner account, follow below steps:

- Login to your emSigner account with valid credentials.
- In the emSigner dashboard, Click on subscription tab in the left navigator. You will be redirected to a subscription page, Click on Add counters button.
- Enter the number of counters required for signer gateway and enter the required details. Click on Purchase button, now you will be redirected to payment gateway page.
- Complete payment. On successful payment, you would receive the email with successful counter upgradation. Please note that you can add counters only after complete usage of counters with an active subscription.

4. Admin panel

4.1 How to generate Authtoken?

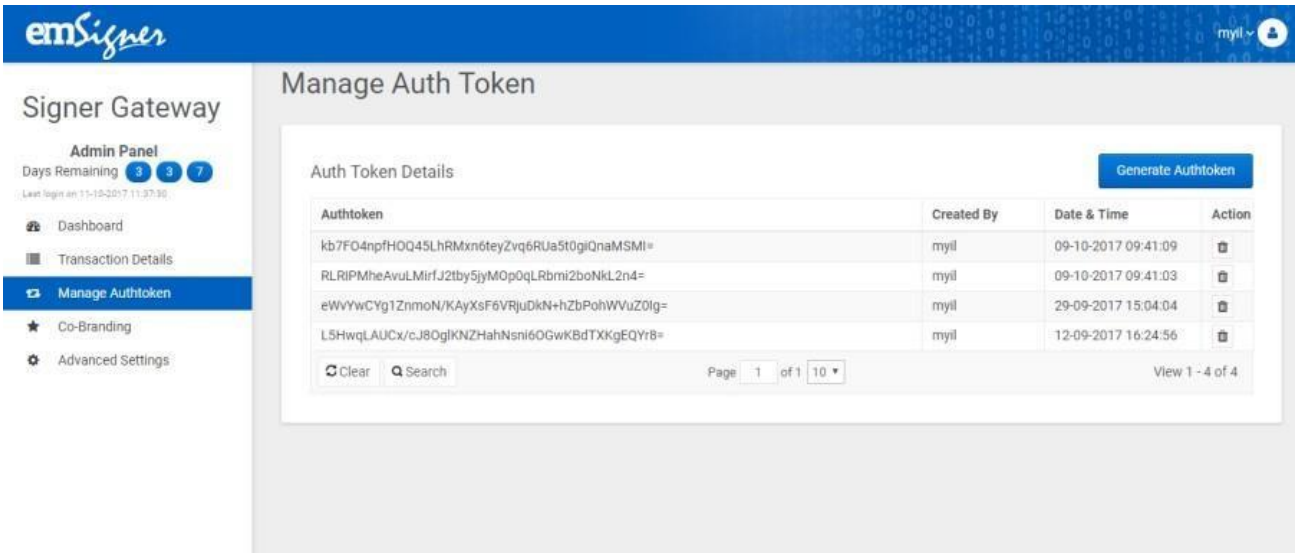
Usually, an Authtoken is a unique identifier of an application requesting access to our service. emSigner would generate a unique Authtoken for each customer to access emSigner signer gateway service. To use the emSigner signer gateway, you'll require the Authentication Token that is uniquely created for your account.

- Log in to emSigner signer gateway account.
- In the emSigner dashboard page.
- Click Manage Authtoken tab in the left navigator.
- This page consists of all the information of Authtoken as below.
- **Authtoken**: Authtoken that is uniquely generated for your account will be displayed here.
- **Created By**: Created by will have the name of the person, who created the Authtoken.
- **Date & Time**: Date and time of the Authtoken created.
- **Action**: On click of **delete** icon in actions column deletes token permanently.

Note:

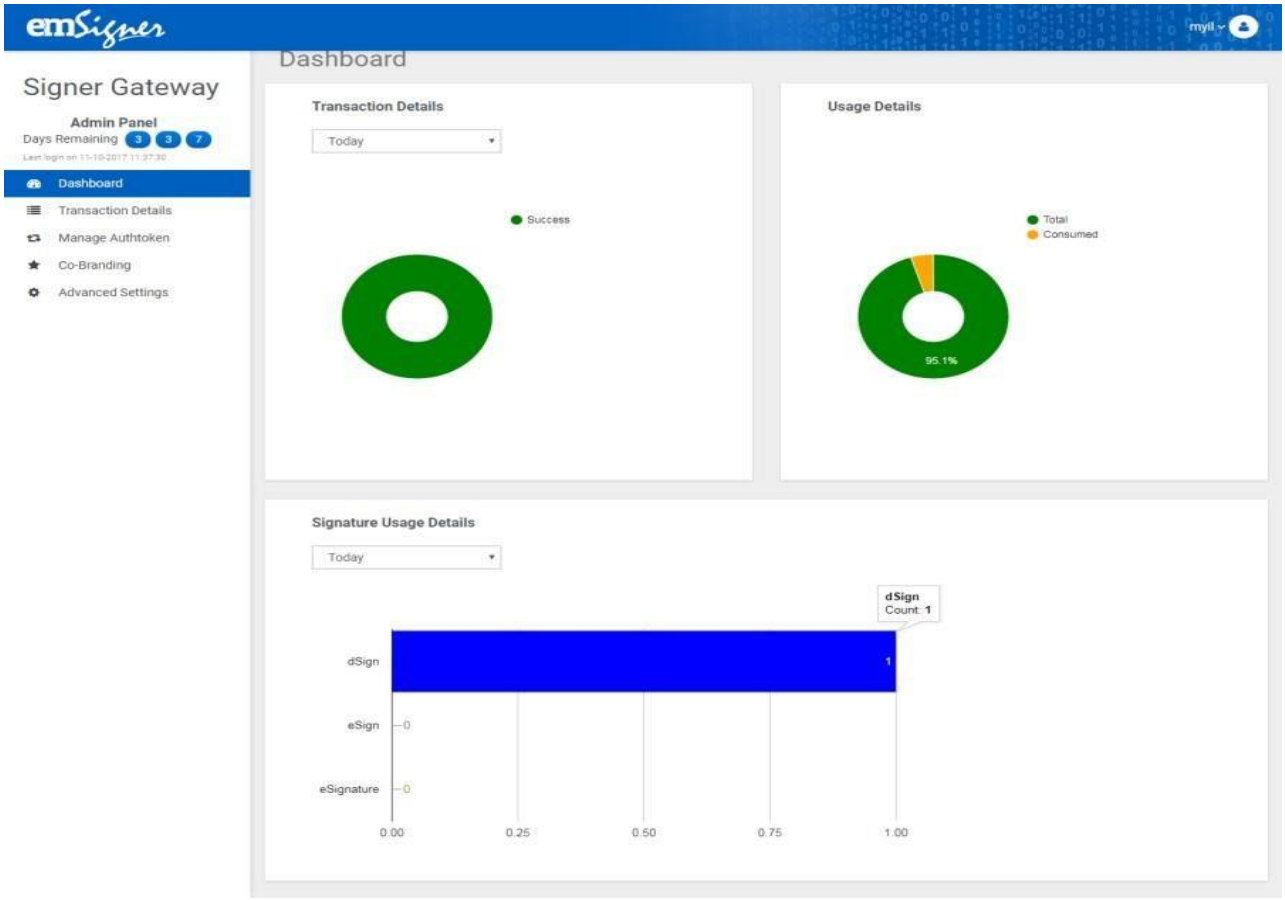
- The Authtoken is user-specific and is a permanent token.
- You can generate multiple Authtokens.
- Removing an Authtoken will inactive the token permanently and any request with the usage of the inactive token will be invalid.
- If you generate new Authtoken, update your program with the new token. The new token has to be replaced for all signer gateway calls.
- The Authtoken of a user's account will become invalid if the user subscription is expired or deactivated.

- Never share your Authtoken and account credential details to anyone. Sharing these details can lead to unauthorized access to emSigner signer gateway.



4.2 How to check my counters usage?

The counter is a variable which is initially set to the number of counters purchased and gets decremented for each transaction request. You can always check emSigner gateway counters usage from your account.

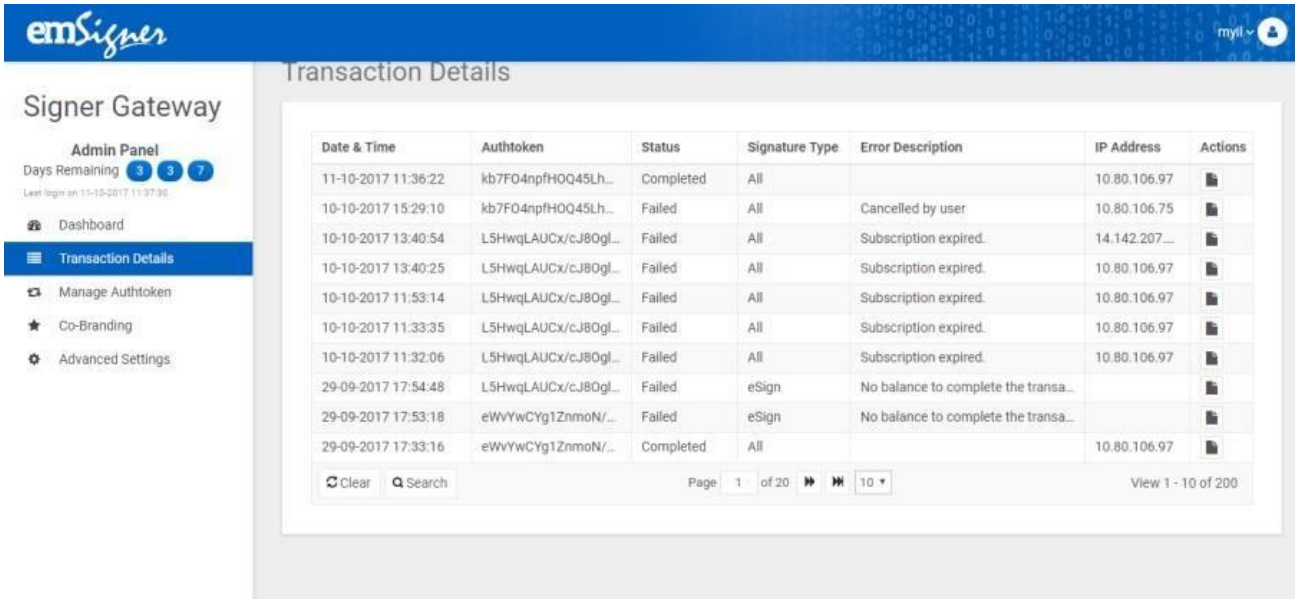


- Login to your emSigner gateway account with valid credentials.
- Check the graphical charts in the dashboard

The dashboard contains all the transaction details by date. By selecting the date user can be able to see the transactions made for that particular day. Usage Details will have all the details like a total number of counters and consumed counters in the pie chart format, by this user can know the counters information and can take the decision accordingly.

4.3 How to check my transaction details?

emSigner tracks all the emSigner gateway transaction request details. You can always check your account.



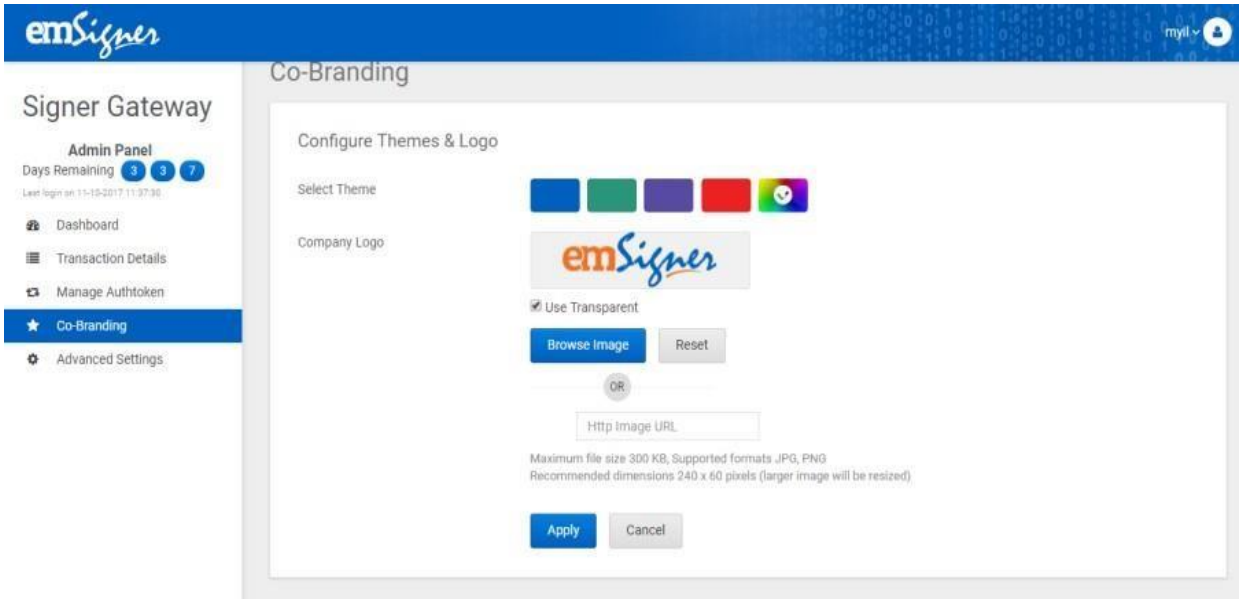
- Login to your emSigner gateway account with valid credentials.
- Click on Transaction details tab, in the left navigator.

Transaction details will consist of below information.

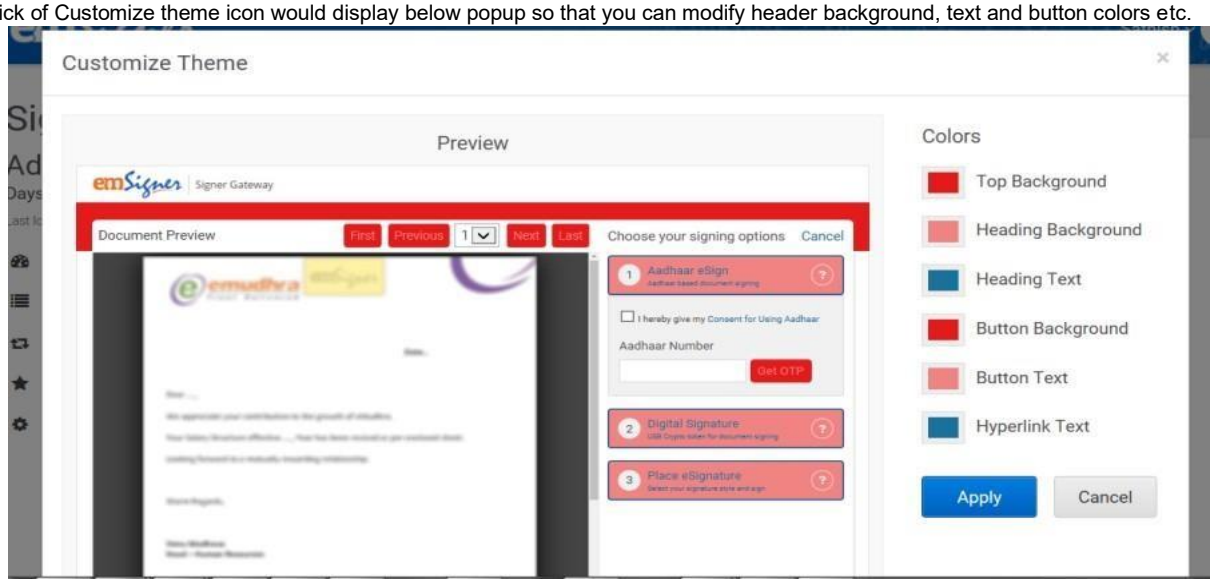
- **Date & Time**: Date and time of the particular transaction.
- **Authtoken**: Authtoken used for the particular transaction.
- **Status**: Status of particular transaction like (completed, initiated, canceled and failed).
- **Signature Type**: Signature type can be dSign and eSignature.
- **Error Description**: Error description will be only for failed and canceled transactions, there will not be any description for a completed transaction.
- **IP Address**: IP Address used for the particular transaction.
- Actions: Actions consists of transaction log details like (User, status, and remarks).

4.4 How can I co-brand signer gateway page?

Brand your account by configuring themes and logo, along with your signing experience. Configuring logo helps users to identify documents coming from your organization. This update adds the ability to customize your account profile. Branding your account is an excellent way to add the look and feel of your organization's brand.



On click of Customize theme icon would display below popup so that you can modify header background, text and button colors etc.



To brand your account:

- Login to your emSigner gateway account with valid credentials.
- Click on Co-branding tab, in the left navigator.

5. ESGS integration

Here enterprises need to be send data to the ESGS though HTTPS POST request. The steps for integrating with ESGS can technically be described as below:

To start off the integration process, follow below steps:

There are five steps of process:

Step 1: Generate unique Session Key for each request

Step 2: Create JSON data and encrypt using unique Session key (Step 1)

Step 3: Generate Hash of data

Step 4: Encrypt generated Hash (Step3) with the unique Session Key (Step1)

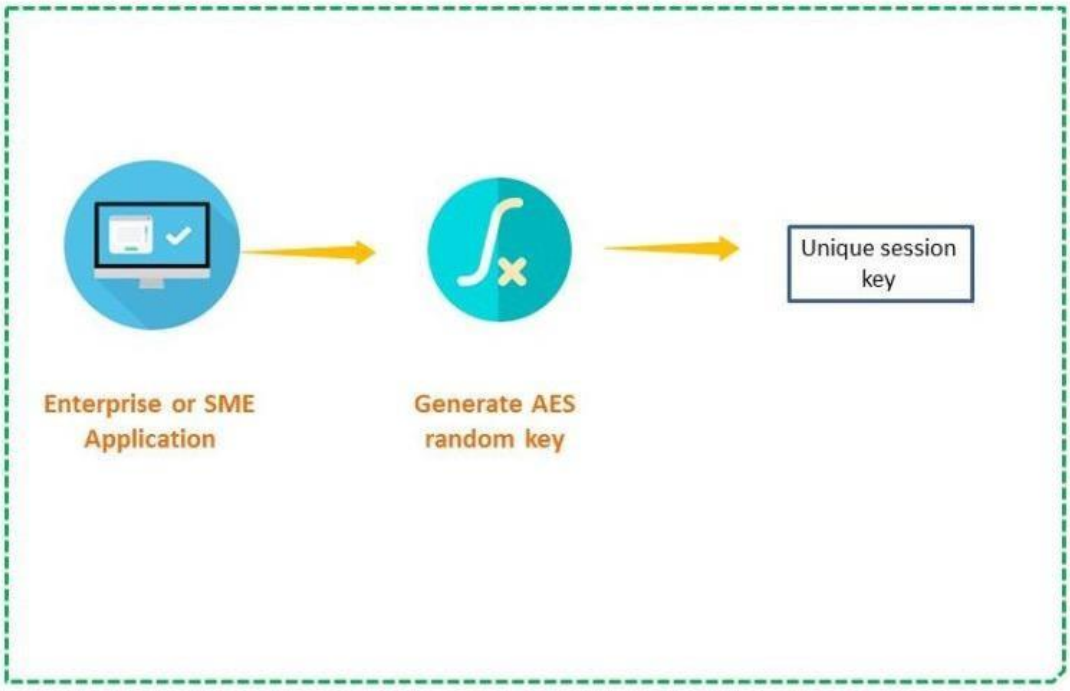
Step 5: Encrypt the unique Session Key (Step 1) using public key of the emSigner

Step 6: HTTPS Post request with step2+step4+step5 output

Step 7: Decrypt encrypted signed data (Step6) with session key (Step1)

5.1 Step1: Generate unique Session Key for each request

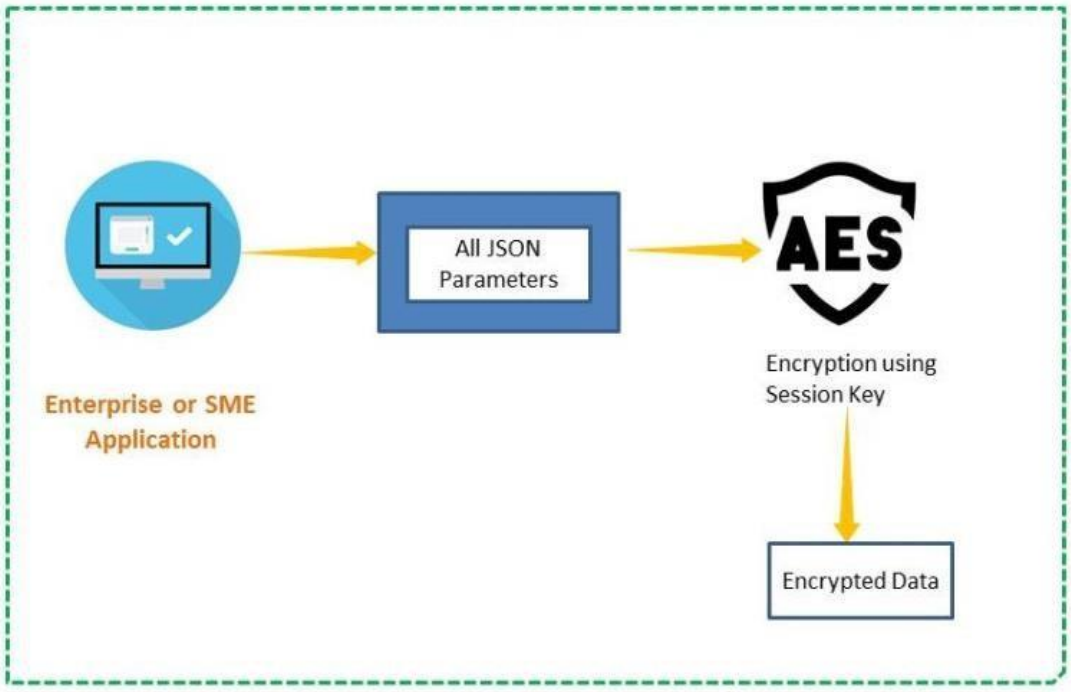
Enterprise needs to generate a random session key for each transaction/request.



5.2 Step2: Create JSON data and encrypt using unique Session key

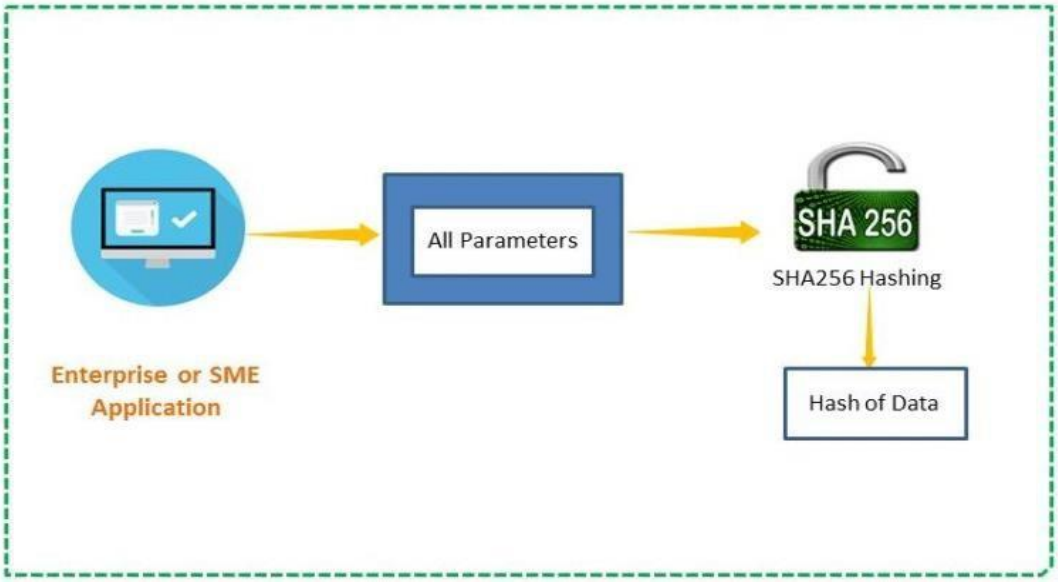
Here you need to convert all the data parameters into JSON format and encrypt that JSON format using AES encryption by unique session key generated in the step1.

AES (acronym of Advanced Encryption Standard) is a symmetric encryption algorithm. AES was designed to be efficient in both hardware and software and supports a block length of 128 bits and key lengths of 128, 192, and 256 bits.



5.3 Step3: Generate Hash of data

Here you need to generate hash of all the data parameters using SHA256 hashing.



5.3.1 Request Post data Input Parameters

JSON Data

```
{
    "Name": "",
    "FileType": "PDF",
    "SelectPage": "",
    "SignaturePosition": "",
    "AuthToken": "",
    "File": "",
    "PageNumber": "",
    "PreviewRequired": true,
    "PagelevelCoordinates": "",
    "CustomizeCoordinates": "",
    "SUrl": "/Success",
    "FUrl": "/Error",
    "CUrl": "/Cancel",
    "ReferenceNumber": "",
    "Enableuploadsignature": true,
    "Enablefontsignature": true,
    "EnableDrawSignature": true,
    "EnableeSignaturePad": false,
    "IsCompressed": false,
    "IsCosign": false,
    "Storetodb": false,
    "IsGSTN": false,
    "IsGSTN3B": false,
    "IsCustomized": false,
    "eSign_SignerId":,
    "TransactionNumber":,
    "SignatureMode": "1,2,3,8,12,14",
    "AuthenticationMode": 1,
    "EnableInitials": false,
    "IsInitialsCustomized": false,
    "SelectInitialsPage": null,
    "InitialsPosition": null,
    "InitialsCustomizeCoordinates": "",
    "InitialsPagelevelCordinates": "",
    "InitialsPageNumbers": "",
    "ValidateAllPlaceholders": false,
    "Searchtext":,
    "Anchor": "Middle",
    "InitialSearchtext": null,
    "InitialsAnchor": null,
    "Reason": null
    "EnableFetchSignerInfo": null,
    }
```

5.3.2 Specifications

Sl No	Parameter	Data Type	Presence	Description
1	Name	String	Mandatory	Pass the name of the signatory which will be displayed along with the signature.
2	FileType	String	Mandatory	Specify Filetype=PDF, If file is PDF format. Filetype=XML, if file is XML format. Filetype=HASH, if file is HASH format. Refer Table 3 Parameters vary with file type. • PDF Parameters • XML & DATA Parameters Filetype=DATA, if file is text format.
3	SignatureMode	Int	Mandatory	This parameter describes the signature mode to be enabled for completing the request. Currently, we support four signature options i.e. dSign, eSign, eSignature and eIDAS. • **dSign**: Requires Crypto Token and valid digital signature issued by licensed Certifying Authorities to digitally sign the request. • **eSign v3**: eMudhra KYC based signing • **eSignature**: eSignature is not valid in India, but it is widely accepted in other countries. • **eSign** v2.1: eSign (Aadhar based signing) is valid ONLY in India.
4	SelectPage	String	Mandatory	Refer Table 1
5	SignaturePosition	String	Mandatory	Signature placeholder specification where the signature must be placed on a page. Refer Table 2

6	AuthToken	String	Mandatory	Usually, an Authtoken is a unique identifier of an application requesting access to our service. emSigner would generate a unique Authtoken for each customer to access emSigner signer gateway service. Pass valid unique Authtoken generated for your account. To generate Authtoken, click here.

7	File	String	Mandatory	Compress the file using 7zip (Optional) and pass compressed base 64 string of file that needs to be signed. If you are compressing the document, make sure you send extra parameter i.e. IsCompressed as TRUE, otherwise FALSE. Compression ratio is high for XML and DATA, whereas for PDF it is only 10% (Varies with size of the document). To know how to compress the file, refer to this section.
8	PageNumber	Int	Mandatory	Pass page numbers separated with a comma, where you wanted to see the signature in the document. **Example:** 1,2 **Note:** This is mandatory if SelectPage is SPECIFY.
9	PreviewRequired	Boolean	Mandatory	By default, this will be **FALSE**. If you want to show preview to the user when redirected to emSigner signer gateway page. Pass **TRUE** if the preview is required before signing the document. Note: Not applicable to dsign

10	PagelevelCoordinates	String	Mandatory	Specify coordinates of page level as PagelevelCoordinates= Pagenumber1,left,top,width,height; Pagenumber2,left,top,width,height **Example:** In 10 pages document you wanted the document to be signed on 1st and 6th page with some co-ordinates then specify PagelevelCoordinates=1,7,10,60,120;6,45,20, 60,120 **Note:** This is mandatory if SelectPage is
				PagelevelCoordinates.
11	CustomizeCoordinates	String	Mandatory	CustomizeCoordinates=**left,top,width,Height** **Example:** In 10 pages document you wanted the document to be signed on particular positions then specify CustomizeCoordinates=7,10,60,120 **Note:** This is mandatory if SignaturePosition is Customize.
12	SUrl	String	Mandatory	Pass success URL, so that after successful signing signer gateway will send a response to that particular URL. The response handling can then be done by you after redirection to this URL.
13	FUrl	String	Mandatory	Pass failure URL, so that when there is a failure, signer gateway will send a response to that particular URL. The response handling can then be done by you after redirection to this URL.

14	CUrl	String	Mandatory	Pass cancel URL, so that when customer cancels the signer gateway process in the middle, signer gateway will send a response to that particular URL. The response handling can then be done by you after redirection to this URL.

15	ReferenceNumber	String	Mandatory	Pass unique random number as a reference number for each transaction request. Maximum of 20 characters. **Example:** 1213 **Note:** • You can pass timestamp as a reference number to create uniqueness. • You are not allowed to pass reference number which has been used for any of the previous transactions.
16	Enableuploadsignature	Boolean	Mandatory	By default this will be **FALSE**, if enabled then the user will have the option to upload handwritten signature in emSigner Signer Gateway page and the same will be affixed on the signed PDF document And using this option will invalidate any signed document sent for signing again by another signatory. Pass **TRUE** if you want to enable upload signature option in the gateway. **Note:** If the user selects signature type as eSignature or combination of that, then at that time any of the parameter i.e. enable eSignature pad/enable draw signature/enable font signature/enable upload signature is mandatory. For dSign, it is not mandatory.

17	Enablefontsignature	Boolean	Mandatory	By default this will be **FALSE**, if enabled then the user will have the option to choose or generate some font based signature in emSigner Signer Gateway page and the same will be affixed on the signed PDF document And using this option will invalidate any signed document sent for signing again by another signatory. Pass **TRUE** if you want to enable font based signatures to be visible in the gateway. **Note:** If the user selects signature type as eSignature or combination of that, then at that time any of the parameter i.e. enable eSignature pad/enable draw signature/enable font signature/enable upload signature is mandatory. For dSign, it is not mandatory.
18	EnableDrawSignature	Boolean	Mandatory	By default this will be **FALSE**, if enabled then the user will have the option to draw signature in emSigner Signer Gateway page and the same will be affixed on the signed PDF document And using this option will invalidate any signed document sent for signing again by another signatory. Pass **TRUE** if you want to enable draw signature option in the gateway. **Note:** If the user selects signature type as eSignature or combination of that, then at that time any of the parameter i.e. enable eSignature pad/enable draw signature/enable font signature/enable upload signature is mandatory. For dSign, it is not mandatory.

19	EnableeSignaturePad	Boolean	Mandatory	By default this will be **FALSE**, if enabled then the user will have the option to connect to third-party signature pads. Currently, we support Topaz signature pad in emSigner Signer Gateway page and the same will be affixed on the signed PDF document And using this option will invalidate any signed document sent for signing again by another signatory. Pass **TRUE** if you want to enable eSignature pad option in the gateway.
20	IsCompressed	Boolean	Optional	If the file size is more than 10mb, you can compress and share the file
21	IsCosign	Boolean	Mandatory	By default, this parameter would contain a **FALSE** value. Set this to **TRUE**, if you want to sign the document multiple times with different signatories. For security reasons, if you have only one signatory signing the document pass as false and also for the last signatory of your workflow/business flow you need to pass false so that document will not be allowed for further signing.

22	Storetodb	Boolean	Mandatory	By default this value would be **FALSE**. Pass it as **TRUE** if you want to save the response data with us. In case of failure you can request us by calling our API's before 48 hours (If Storetodb=TRUE). Refer to Transaction Request API section, for more info. Refer to Signed Data Request API section, for more info.

23	IsGSTN	Boolean	Mandatory	By default this value will be **FALSE**. Pass **TRUE** if you want to sign GSTR1 data and Pass "IsGSTN3B" as FALSE. Pass file value as Base 64 of (GSTR1 JSON data)
24	IsGSTN3B	Boolean	Mandatory	By default this value will be **FALSE**. Pass **TRUE** if you want to sign GSTR3B data and Pass "IsGSTN" as FALSE. Pass file value as Base 64 of (Part A~Part B)
25	IsCustomized	Boolean	Optional	If enabled, the signature can be dragged and dropped in desired location.
26	eSign_SignerId	String	Optional	If eSign v3 is enabled, the signerId will be prefilled
27	TransactionNumber	String	Optional	A unique transaction ID of the request.
28	AuthenticationMode	Int	Mandatory	• 1 - OTP (One Time Password) • 2 - Biometric • 3 - Iris • 4 - Face(Applicable for mobile)
29	EnableInitials	Boolean	Optional	If enabled initials are displayed in the preview page

30	IsInitialsCustomized	Boolean	Optional	If enabled, the initials can be customised
31	SelectInitialsPage	String	Conditional Mandatory	Pass page numbers separated with a comma, where you wanted to see the signature in the document.
32	InitialsPosition	String	Conditional Mandatory	Refer Table 2
33	InitialsCustomizeCoordinates	String	Conditional Mandatory	If enabled, the initials coordinates can be customised
34	InitialsPagelevelCordinates	String	Conditional Mandatory	If enabled, the initials page level coordinates can be customised accordingly.
35	InitialsPageNumbers	String	Conditional Mandatory	If enabled, the initials page number can be customised accordingly.
36	ValidateAllPlaceholders	Boolean	Conditional Mandatory	If not enabled, the signature need not be selected for signing. It will be appended with a single click.
37	Searchtext	String	Optional	Desired text in the PDF where the signature needs to be appended.
38	Anchor	String	Conditional Mandatory	If "Searchtext" is enabled, this can be specified by choosing either • Top • Middle • Bottom
39	InitialSearchtext	String	Optional	Desired text in the PDF where the signature needs to be appended.
40	InitialsAnchor	String	Conditional Mandatory	If "InitialSearchtext" is enabled, this can be specified by choosing either • Top • Middle • Bottom
41	Location	String	Optional	Location of the signature
42	Reason	String	Optional	Purpose of signing

43	DynamicContent	String	Optional	Applicable only for V2. Getting used in case of adding dynamic contents in PDF appearance. **Value** - Base64 value for the below list [{"Key":"","Value":""},{"Key":"","Value":""}] Note: **DateTime** is keyword in case to get the Date Time
44	EnableFetchSignerInfo	Boolean	Optional	Applicable only for V2(Aadhar based signing). Fetches the user details. By default, this value will be **FALSE**. Pass **TRUE** if you want to enable EnableFetchSignerInfo.

5.3.3 Enum

SelectPage

Sl No	Name	Description
1	ALL	if you need a signature on all the pages of the document
2	FIRST	if you need a signature on the only first page of the document
3	EVEN	if you need a signature on all even pages of the document. For example: If the document consists of 10 pages then second, fourth, sixth, eighth and tenth pages contain a signature
4	LAST	if you need a signature on only last page of the document
5	ODD	if you need a signature on all odd pages of the document. For example: If the document consists of 10 pages then first, third, fifth, seventh and ninth pages contain a signature
6	SPECIFY	if you want to specify page numbers. For specifying page level coordinates, you need to pass additional parameter i.e. PageNumber which is explained below
7	PAGE LEVEL	if you want to specify page coordinates. For specifying page level coordinates, you need to pass additional parameter i.e. PagelevelCoord inates which is explained below

Table 1

Signature Position

Sl No	Name	Description
1	Top-Left	Makes signature appearance to be on Top Left of the document.
2	Top-Center	Makes signature appearance to be on Top Center of the document.
3	Top-Right	Makes signature appearance to be on Top Right of the document.
4	Middle-Left	Makes signature appearance to be on Middle Left of the document.
5	Middle-Right	Makes signature appearance to be on Middle Right of the document.
6	Middle-Center	Makes signature appearance to be on Middle Center of the document.
7	Bottom-Left	Makes signature appearance to be on Bottom Left of the document.
8	Bottom-Right	Makes signature appearance to be on Bottom Right of the document.
9	Bottom-Center	Makes signature appearance to be on Bottom Center of the document.
10	Customize	If you want to specify page coordinates. For specifying customize coordinates, you need to pass additional parameter i.e., CustomizeCoordinates

Table 2

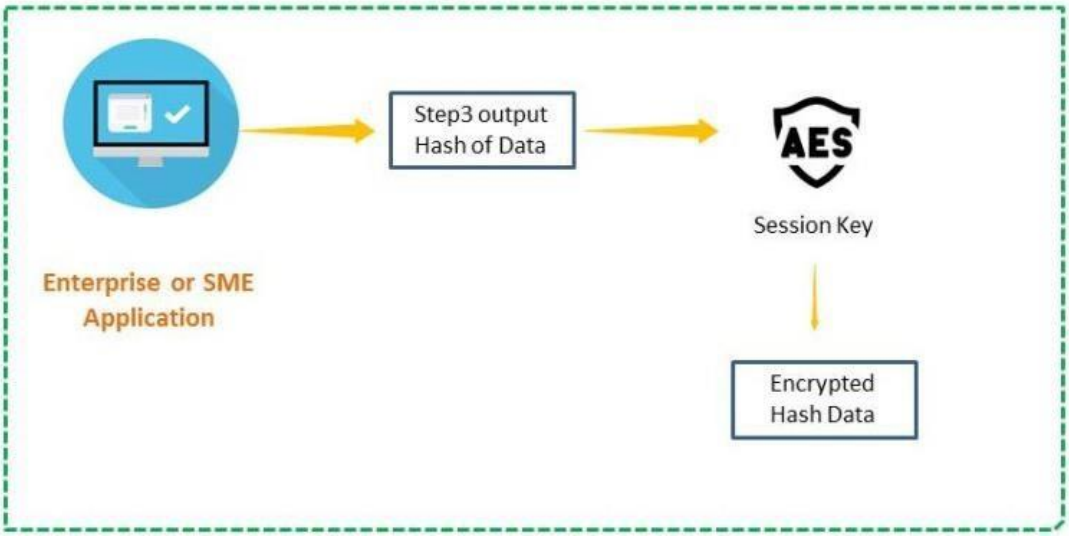
Hash Signing

Sl No	Name	Data Type	Presence	Description
1	DocumentName	String	Mandatory	Name of the document
2	DocumentURL	String	Mandatory	Document url
3	DocumentHash	String	Mandatory	Hash of the document
4	Document Details	String	Mandatory	[{'DocumentName':'','DocumentURL':'','DocumentHash':''}]
5	File	String	Mandatory	Base64 of encoded Document Details

Table 3

5.4 Step4: Encrypt generated Hash (Step3) with the unique Session Key (Step1)

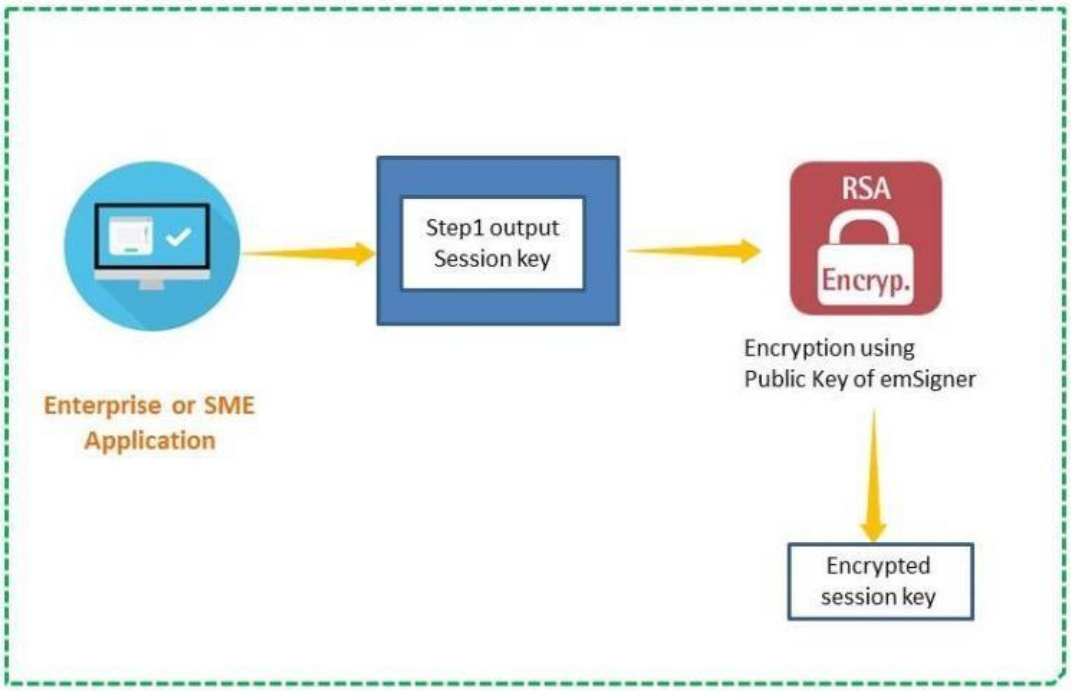
Here you need to encrypt the hash of data obtained from step3 by unique session key generated in the step1.



5.5 Step5: Encrypt the unique Session Key (Step 1) using public key of the emSigner

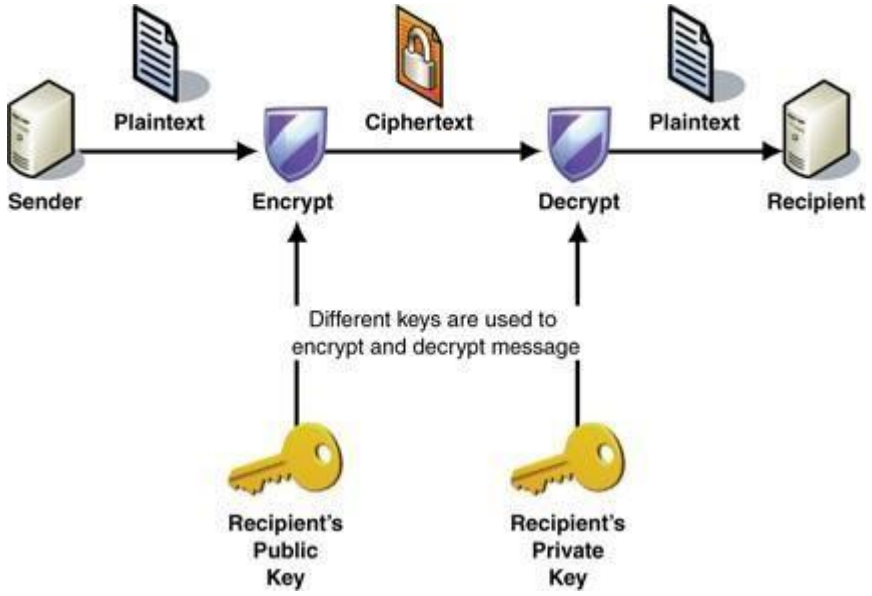
Need to encrypt the session key generated in the step1 with public key provided by ESGS (emSigner).

For encryption you need to use RSA algorithm.



In RSA cryptography, both the public and the private keys can encrypt a message; the opposite key from the one used to encrypt a message is used to decrypt it.

RSA encrypted session key using public key provided by emSigner.



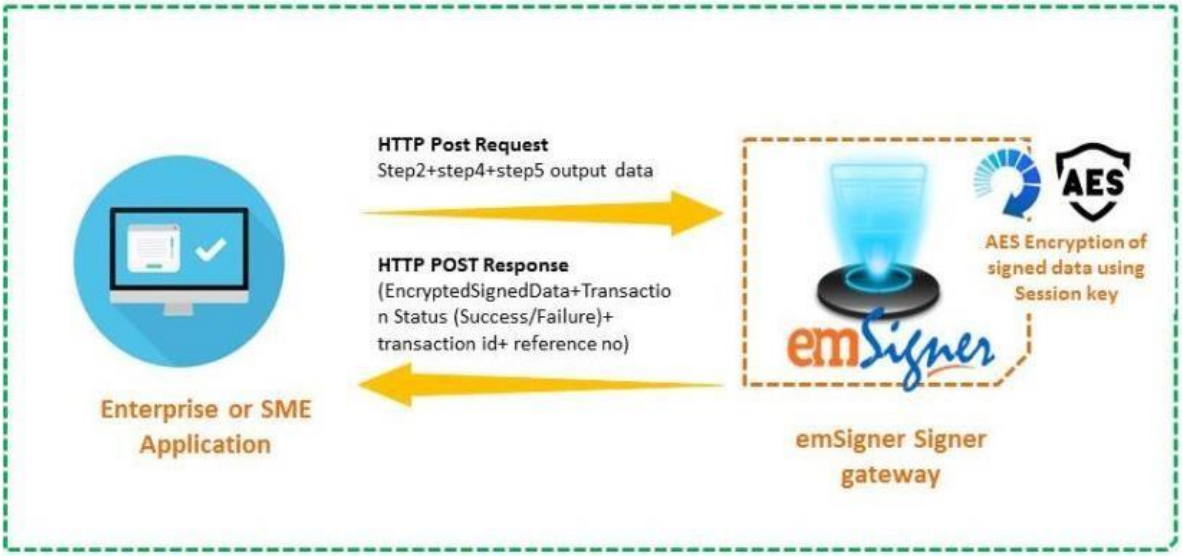
5.6 Step6: HTTPS Post request with step2+step4+step5 output

Here corporate has to send Step2, Step4 and Step5 output to ESGS URL through HTTPS POST.

Test URL:

"https://demosignergateway.emsigner.com/eMsecure/V3_0/Index"

Production URL: https://emgateway.emsigner.com/



Please refer below request parameters.

5.6.1 Request Parameters

Parameter	Data Type	Description
Parameter1*	String	Pass step5 output
Parameter2*	String	Pass step2 output
Parameter3*	String	Pass step4 output

- Once ESGS receives the request (Step2+step4+step5 output), then it authenticates by decrypting the all the values and compares them to check if there is any man in the middle attack and authorizes the request.
- On successful authentication, redirects to the ESGS gateway page where customer can view the data based on the request type.
- On successful authentication, ESGS generates signed data/document with signed data and sends HTTPS POST response to the corporate success URL with Encrypted Signed data, Transaction status, Transaction number and reference number.
- Here signed document is compressed using 7-zip (if the file is of larger size) and sent as a response. Signed data will be encrypted using AES encryption with session key received in step 1 output.
- If authentication fails, customer will be redirected to the failure URL and error message will be displayed. Please refer error messages listed below.

5.6.2 Response Parameters

Parameter	Data Type	Description
ReturnStatus	String	Returns Success if request is done successfully otherwise Failure
ErrorMessage	String	If ReturnStatus is "Failure" it will return an error message. If this is empty then request is a success. Check below error messages.
Returnvalue	String	Returns Encrypted Signed data, if signing is done successfully otherwise Failure
Transactionnumber	String	Returns unique transaction number for each request. To identify each new transaction in the ESGS Database, a unique identifier is created every time. This identifier is known as the Transaction number.
Referencenumber	String	Returns same reference number which has raised the request.

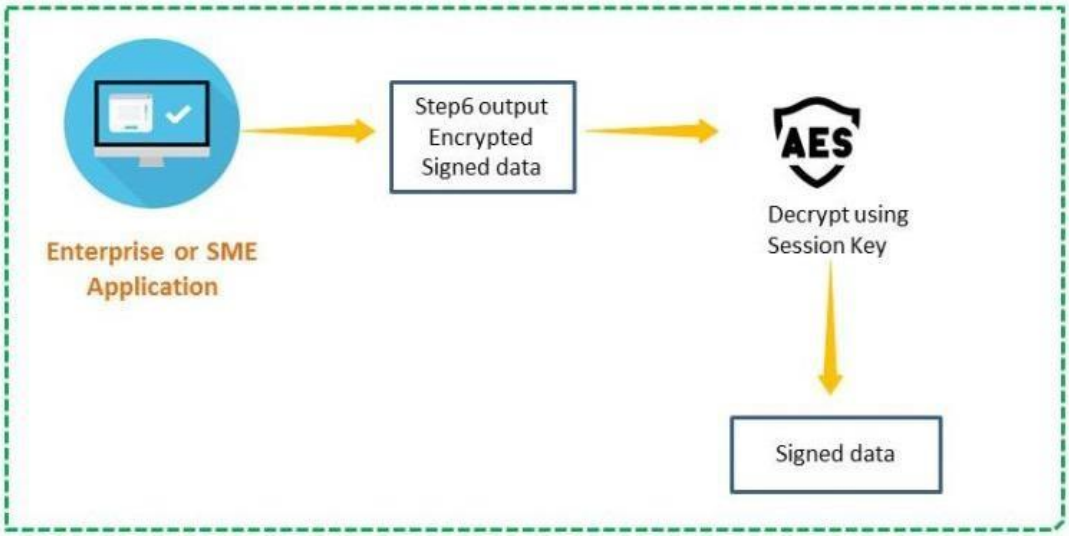
SignerInfo	String	Returns Encrypted Signed data that contains signer details, if signing is done successfully otherwise Failure

Error Messages

Error Code	Error Message	Description
104	Invalid Authtoken	When invalid Authtoken is entered
105	You are not subscribed to this VAS item.	When No VAS feature is subscribed.
106	Cancelled by user	When client cancels the request
107	Invalid parameters	When any parameter is empty
108	You cannot use reference number which is already used in previous transactions.	When reference number is already used in previous transactions.
109	Invalid Request	When decryption of data failed
110	emSigner application is required to digitally sign the documents using token-based signature.	When emSigner utility is not installed
111	emSigner application is required to digitally sign the documents using token-based signature.	When utility not running in the system
112	emSigner application is required to digitally sign the documents using token-based signature.	When port number is mismatch
113	Please update the emSigner utility for new updates	If Utility used is not the latest version.
114	eSignature is not applicable for XML and Data signing.	When trying to do eSignature for XML and Data files.
115	Only .jpg,.jpeg and .png files can be set as Signature Image.	While uploading signature image other than jpg, jpeg and png files Allowed file types are .jpg, .jpeg and .png
116	You can upload maximum of 200KB and minimum of 100KB file size.	While uploading big size file Ensure that the size of file should be between (100KB - 200KB)

117	Please connect the eSignature pad to sign and capture the signature image	When eSignature Pad is not connected

5.7 Step7: Decrypt encrypted signed data (Step6) with session key (Step1)



Now decrypt the Encrypted Signed data with session key generated in the Step1.

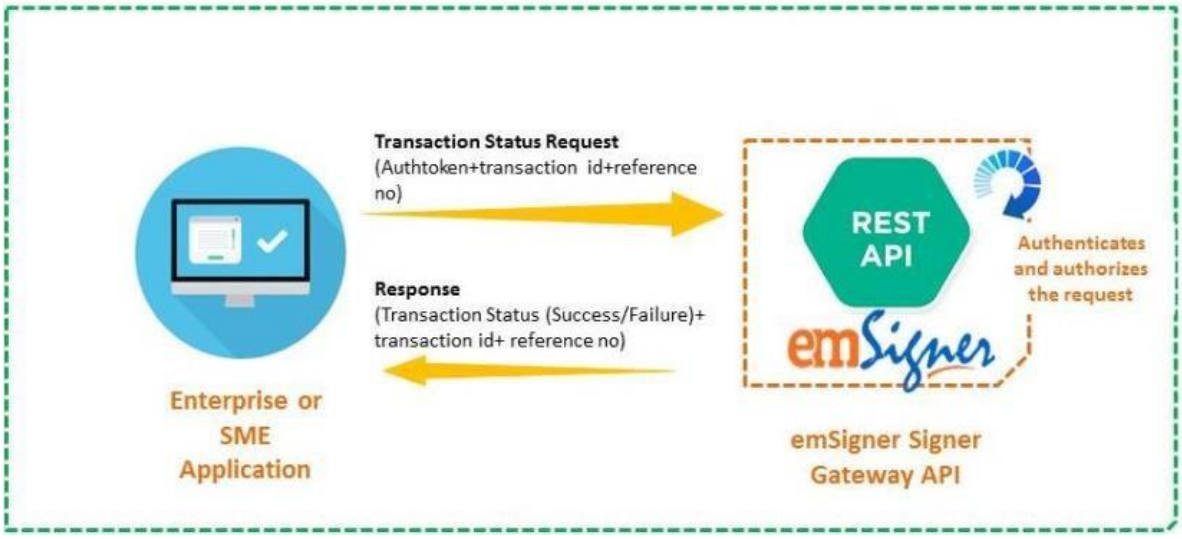
6. API Methods

In computer programming, an application programming interface (API) is a set of subroutine definitions, protocols, and tools for building application software. An API specification can take many forms, but often includes specifications for routines, data structures, object classes, variables or remote calls.

There are two API methods in signer gateway they are:

- Transaction Status Request API
- Signed Data Request API

6.1 Transaction Status Request API



Method Name: TransactionStatusRequest

Purpose: You can use the "TransactionStatusRequest" method to retrieve transaction status i.e. Completed/Failed/initiate.

Type of Method: GET

TEST URL: https://demosignergateway.emsigner.com/api/TransactionStatusRequest?Authtoken=cbFdZK9esvBzZQ1QynL7IjKb7t92K%2BB4SoEzVA%3D&Transactionnumber=1213&Referencenumber=78123897 **Production URL**: https://emgateway.emsigner.com/

For calling this API, you need to send below request parameters:

6.1.1 Request data Input Parameters

Parameter	Data Type	Description
AuthToken*	String	Usually, an Authtoken is a unique identifier of an application requesting access to our gateway. Generate Authtoken (To generate Authtoken, click here) and URL encode of the generated Authtoken. Pass valid URL encoded Authtoken. Eg: cbFdZK9esvBzZQ1QynL7IjKb7t92K% 2BB4SoEzVA%3D
Transactionnumber*	String	Pass unique transaction number received in the Step6. To identify each new transaction in the ESGS Database, a unique identifier is created every time. This identifier is known as the Transaction number.

Referencenumber*	AlphaNumeric	Bank's unique internal identification reference for the mandate. This is usually the reference of the mandate in the internal systems of the bank. Pass unique random number as reference number for each transaction request. Example: 1213 **Note**: You can pass timestamp as a reference number to create uniqueness. You are not allowed to pass reference number which has been used for any of the previous transactions. Eg: 00000000000110|

6.1.2 Response Parameters

Parameter	**Data Type**	**Description**
IsSuccess	Boolean	Returns TRUE if request is success otherwise FALSE
Messages	Collection of String	If IsSuccess is true it will return “Transaction Status successfully retrieved”.
Value	Collection of Info	
Transactionnumber	String	Returns transaction number that is received in request.
Referencenumber	String	Returns reference number that is received in request.
Status	String	Returns Completed, if signing is done successfully otherwise Failed. Returns Initiate, if user just initiated and left in the middle.
ErrorMessage	Collection of String	Returns error message if transaction status retrieval failed. Refer error messages section 1.9

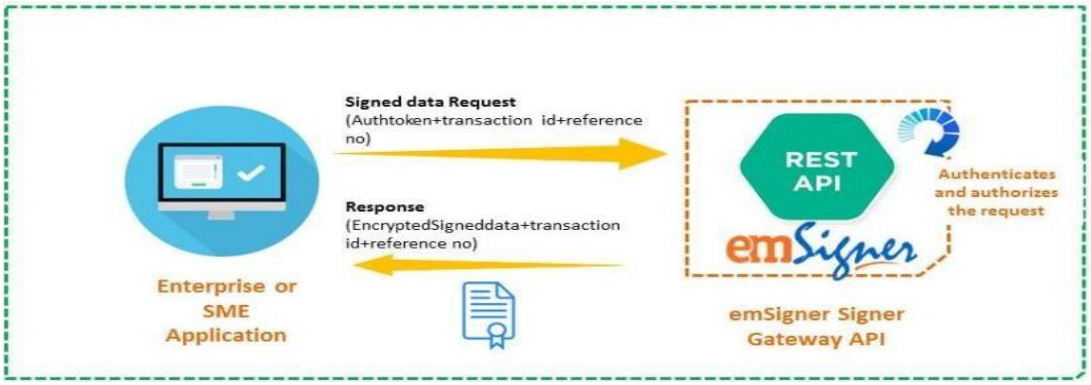
6.1.3 Response JSON Format Sample:


```
{

        "IsSuccess": true,
        "Messages": [
                "Status Information Found"
        ],
        "Value": {
                "TransactionNumber": "",
                "ReferenceNumber": "",
                "Status": "",
                "ErrorMessage": ""
        }

}
```


6.1.4 Error Messages

Error Code	Error Message	Description
201	Invalid parameters	When any parameter is empty/invalid.
202	Invalid Operation	If API call is invalid

6.2 Signed Data Request API



Method Name: SignedDataRequest

Purpose: You can use the "SignedDataRequest" method to retrieve signed data.

Type of Method: GET

TEST URL: https://demosignergateway.emsigner.com/api/SignedDataRequest?Authtoken=cbFdZK9esvBzZQ1QynL7IjKb7t92K%2BB4SoEzVA%3D&Transactionnumber=1213&Referencenumber=78123897

Production URL: https://emgateway.emsigner.com/

For calling this API, you need to send below request parameters:

6.2.1 Request data Input Parameters

Parameter	Data Type	Description
Authtoken*	String	Usually, an Authtoken is a unique identifier of an application requesting access to our gateway. Generate Authtoken (To generate Authtoken, click here) and URL encode of the generated Authtoken. Pass valid URL encoded Authtoken. Eg: cbFdZK9esvBzZQ1QynL7IjKb7t92K% 2BB4SoEzVA%3D
Transactionnumber*	String	Pass unique transaction number received in the Step6. To identify each new transaction in the ESGS Database, a unique identifier is created every time. This identifier is known as the Transaction number.
Referencenumber*	AlphaNumeric	Bank's unique internal identification reference for the mandate. This is usually the reference of the mandate in the internal systems of the bank. Pass unique random number as reference number for each transaction request. Example: 1213 **Note**: You can pass timestamp as a reference number to create uniqueness. You are not allowed to pass reference number which has been used for any of the previous transactions. Eg: 00000000000110|

6.2.2 Response Parameters

Parameter	Data Type	Description
IsSuccess	Boolean	Returns TRUE if request is success otherwise FALSE
Messages	Collection of String	If IsSuccess is true it will return "Signed Data retrieved successfully".
Value	Collection of Info	
Status	String	Returns Completed, if signed data is retrieved successfully otherwise Failed.
Transactionnumber	String	Returns transaction number that is received in request.
Referencenumber	String	Returns reference number that is received in request.
SignedData	Byte 64 String	Returns Encrypted Signed DATA, if signing is done successfully otherwise Failure. For decrypting the signed DATA refer step7
ErrorMessage	Collection of String	Returns error message if signed data retrieval failed. Refer error messages section 2.9

Response JSON Format Sample:

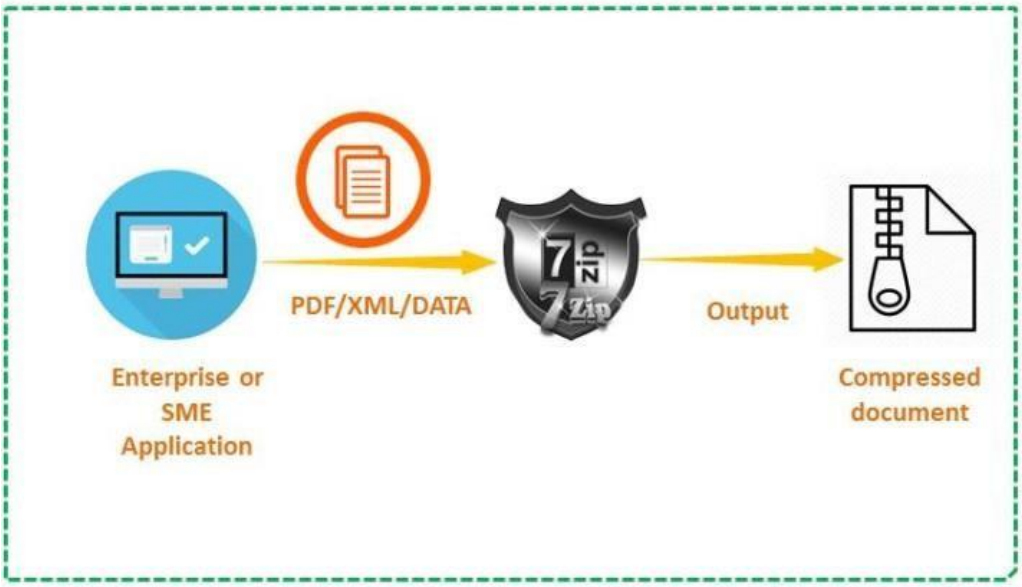

```
{
"IsSuccess": true, "Messages": [
"Signed data found successfully"
],
"Value": {
"Status": "Completed",
"TransactionNumber": "ESG20171213132055259", "ReferenceNumber": "REFDB182",
"SignedData": "Y9nCZh6XiJGW8ZQRPn8kDXr2JDBTzBRz4MT366MCfK7KX+ac9uGMWJjcPqVDRTO4PKY+ZvQT382RQxthT
/qyFDJiiM09NifWF6kk9Ti/IrsqIzOOpOfSD/0CCvBAvpNZ26ywyWGD+ ", "ErrorMessage": ""
}
}
```


6.2.3 Error Messages

Error Code	Error Message	Description
204	Invalid Authtoken	When invalid Authtoken is entered
205	Invalid parameters	When any parameter is empty
206	Invalid Operation	If API call is invalid
207	No signed data found	If signed data is not available for the account

7. How to compress document

In this, we will show you how to compress a file or folder into a .zip file using 7-zip. Compression reduces the size of an application or document for storage or transmission. Compressed files are smaller, download faster, and easier to transport. This is recommended before transferring a file of large size over the internet, so that request can be easily handled. Compression ratio is high for XML and DATA, whereas for PDF it is only 10% (Varies with size of the document).



8. How to de-compress zip file

In this, we will show you how to de-**compress** a *.zip* file into a **file** or folder using 7-zip. Decompression or expansion restores the document or application to its original size.

9. High-end security

ESGS is secured with different validation process like Authtoken is used for validating the account and only the registered account holder can generate Authtoken; https post keeps your information safe and secure.

The ESGS uses Hypertext Transfer Protocol Secure (HTTPS). Using HTTPS, the computers agree on a "code", and then they scramble the messages using that "code" so that no one in between can read them. This keeps your information safe from hackers.

The cipher AES-256 is used in encryption and decryption of posted data. It's considered among the top ciphers. In theory, it's not crack able, since the combinations of keys are massive.

Breaking a symmetric 256-bit key by brute force requires 2128 times more computational power than a 128-bit key. Fifty supercomputers that could check a billion (1018) AES keys per second (if such a device could ever be made) would, in theory, require about 3×1051 years to exhaust the 256-bit key space.

RSA used is a cryptosystem for public-key encryption, and is widely used for securing sensitive data, particularly when being sent over an insecure network such as the Internet. In RSA cryptography, both the public and the private keys can encrypt a message; the opposite key from the one used to encrypt a message is used to decrypt it. This attribute is one reason why RSA has become the most widely used asymmetric. A key aspect of cryptographic hash functions SHA256 is their collision resistance: nobody should be able to find two different input values that result in the same hash output.

Sample codes for ESGS integration

9.1 Step 1: Generate unique Session Key for each request

.NET Example code:

```
public static byte[] GetNewSessionKey()
{
using (Aes myAes = Aes.Create("AES"))
{
myAes.KeySize = 256; myAes.GenerateKey();  return myAes.Key;
}
}
```

Java Example code:

```
String JCE_PROVIDER = "BC"; public byte[] GenerateSessionKey() throws NoSuchAlgorithmException,
NoSuchProviderException { Security.addProvider(new org.bouncycastle.jce.provider.BouncyCastleProvider());
KeyGenerator kgen = KeyGenerator.getInstance("AES", JCE_PROVIDER); kgen.init(256);
SecretKey key = kgen.generateKey();
byte[] symmKey = key.getEncoded();  return
symmKey;
}
```

9.2 Step 2: Create JSON data and encrypt using unique Session key (Step 1)

You can use below code to encrypt JSON data:

.NET Example code:

```
public static byte[] EncryptDataAES(byte[] toEncrypt, byte[] key)
{
```

```
IBufferedCipher cipher = CipherUtilities.GetCipher("AES/ECB/PKCS7");  cipher.Init(true, new KeyParameter(key)); int outputSize =
cipher.GetOutputSize(toEncrypt.Length);  byte[] tempOP = new byte[outputSize]; int processLen = cipher.ProcessBytes(toEncrypt, 0,
toEncrypt.Length, tempOP, 0);  int outputLen = cipher.DoFinal(tempOP, processLen); byte[] result = new byte[processLen + outputLen];
System.Array.Copy(tempOP, 0, result, 0, result.Length);  return result;
}
```

Java Example code:

```
public byte[] EncryptUsingSessionKey(byte[] skey, byte[] data) throws InvalidCipherTextException {
        PaddedBufferedBlockCipher cipher = new PaddedBufferedBlockCipher(new AESEngine(), new
        PKCS7Padding()); cipher.init(true, new KeyParameter(skey));
        //

        int outputSize = cipher.getOutputSize(data.length); byte[] tempOP
        = new byte[outputSize]; int processLen =
        cipher.processBytes(data, 0, data.length, tempOP, 0); int
        outputLen = cipher.doFinal(tempOP, processLen); byte[] result =
        new byte[processLen + outputLen]; System.arraycopy(tempOP, 0,
        result, 0, result.length); return result;
    }
```

9.3 Step 3: Generate Hash of data

.NET Example code:

```
public static byte[] Generatehash256(string text)
      {
         byte[] message = Encoding.UTF8.GetBytes(text);
         SHA256Managed hashString = new
         SHA256Managed(); string hex = ""; var
         hashValue =
         hashString.ComputeHash(message); return
         hashValue;
      }
```

Java Example code:

```
public byte[] GenerateSha256Hash(byte[] message) {
        String algorithm = "SHA-256";
        String SECURITY_PROVIDER = "BC";

        byte[] hash = null;

        MessageDigest digest;
        try {
            digest = MessageDigest.getInstance(algorithm,
            SECURITY_PROVIDER); digest.reset(); hash = digest.digest(message);
        } catch (Exception e) {
            e.printStackTrace();
        }
        return hash;
    }
```

9.4 Step 4: Encrypt generated Hash (Step3) with the unique Session Key (Step1)

9.5 Step 5: Encrypt the unique Session Key (Step 1) using public key of the emSigner

You can use below code to encrypt the session key using RSA algorithm:

.NET Example code:

```
public static string EncryptWithPublicKey(byte[] stringToEncrypt)
      {
          X509Certificate2 certificate; certificate = new
          X509Certificate2(AppDomain.CurrentDomain.BaseDirectory +
          "\\certificate.cer"); byte[] cipherbytes =
          Convert.FromBase64String(Convert.ToBase64String(stringToEncrypt));
          RSACryptoServiceProvider rsa = (RSACryptoServiceProvider)certificate.PublicKey.Key;
          byte[] cipher = rsa.Encrypt(cipherbytes, false); return
          Convert.ToBase64String(cipher);
      }
```

Java Example code:

```
public byte[] EncryptUsingPublicKey(byte[] data) throws IOException, GeneralSecurityException, Exception
{
        // encrypt the session key with the public key

        Security.addProvider(new org.bouncycastle.jce.provider.BouncyCastleProvider());
        PublicKey publicKey = null; byte[] bytevalue= readBytesFromFile("Give
        certificate.cer filepath ");
        //String certificatedata = "";
        CertificateFactory certificateFactory = CertificateFactory.getInstance("X509");
        //byte[] bytevalue = Base64.decode(certificatedata);
        InputStream streamvalue = new ByteArrayInputStream(bytevalue);

        java.security.cert.Certificate certificate = certificateFactory.generateCertificate(streamvalue);
        publicKey = certificate.getPublicKey();
        Cipher pkCipher = Cipher.getInstance("RSA/ECB/PKCS1Padding",
        "BC"); pkCipher.init(Cipher.ENCRYPT_MODE, publicKey); byte[]
        encSessionKey = pkCipher.doFinal(data);
// System.out.println("ENC SESSION KEY : "+ encSessionKey.length);
        return encSessionKey;
    }
```

9.6 Step 6: HTTPS Post request with step2+step4+step5 output

9.7 Step 7: Decrypt encrypted signed data (Step6) with session key (Step1)

You can use below code to decrypt the signed data:

.NET Example code:

```
public static byte[] DecryptJsonDataAES(byte[] toDecrypt, byte[] key)
      {
         IBufferedCipher cipher =
         CipherUtilities.GetCipher("AES/ECB/PKCS7"); cipher.Init(false, new
         KeyParameter(key)); byte[] output = cipher.DoFinal(toDecrypt);
         string plainText =
         Encoding.UTF8.GetString(output); return output;
      }
```

Java Example code:

```
public static byte[] decryptText1(byte[] text, byte[] key) throws Exception
{
byte[] dec_Key= org.apache.commons.codec.binary.Base64.decodeBase64(key);
SecretKey dec_Key_original =new SecretKeySpec(dec_Key,0,dec_Key.length,"AES");
byte[] decodedValue =org.apache.commons.codec.binary.Base64.decodeBase64(text);
Cipher aesCipher = Cipher.getInstance("AES");
aesCipher.init(Cipher.DECRYPT_MODE,
dec_Key_original); byte[] bytePlainText =
aesCipher.doFinal(decodedValue); return bytePlainText;
}
```

10. Code to Compress document

.NET Example code:

```
public string CompressFileLZMA(string InputFileName)
    {
        string TransactionID = DateTime.Now.ToString("yyyyMMddHHmmssfff"); string OutputFileName =
        System.Web.Configuration.WebConfigurationManager.AppSettings["filepath"] + TransactionID + "_" + Path.
GetFileName(InputFileName.Split('.')[0]).ToString(); Int32
        dictionary = 1 << 23; Int32 posStateBits = 2;
        Int32 litContextBits = 3; // for normal files
        // UInt32 litContextBits = 0; // for 32-bit data
        Int32 litPosBits = 0;
        // UInt32 litPosBits = 2; // for 32-bit data
        Int32 algorithm = 2;
        Int32 numFastBytes = 128;

        string mf = "bt4"; bool eos =
        true; bool stdInMode = false;
        SevenZip.Sdk.CoderPropId[] propIDs =  {
            SevenZip.Sdk.CoderPropId.DictionarySize,
            SevenZip.Sdk.CoderPropId.PosStateBits,
            SevenZip.Sdk.CoderPropId.LitContextBits,
            SevenZip.Sdk.CoderPropId.LitPosBits,
            SevenZip.Sdk.CoderPropId.Algorithm,
            SevenZip.Sdk.CoderPropId.NumFastBytes,
            SevenZip.Sdk.CoderPropId.MatchFinder,
            SevenZip.Sdk.CoderPropId.EndMarker
        };

        object[] properties = {
            (Int32)(dictionary),
            (Int32)(posStateBits),
            (Int32)(litContextBits),

            (Int32)(litPosBits),
            (Int32)(algorithm),
            (Int32)(numFastBytes), mf,
            eos


        };

        using (FileStream inStream = new FileStream(InputFileName, FileMode.Open))
        {
            using (FileStream outStream = new FileStream(OutputFileName, FileMode.Create))
```

```
            {
                SevenZip.Sdk.Compression.Lzma.Encoder encoder = new
                SevenZip.Sdk.Compression.Lzma.Encoder(); encoder.SetCoderProperties(propIDs, properties);
                encoder.WriteCoderProperties(outStream); Int64 fileSize; if (eos || stdInMode) fileSize = -1;
                else fileSize = inStream.Length; for (int i = 0; i <
                8; i++) outStream.WriteByte((Byte)(fileSize >> (8
                * i)));
                encoder.Code(inStream, outStream, -1, -1, null);
            }
          }
          return OutputFileName;
        }
```

11. Code to de-compress document

You can use below code to decompress the compressed file(.zip):

.NET Example code:

```
public string DecompressFileLZMA(string InputFileName)
      {
          string FileName = DateTime.Now.ToString("yyyyMMddHHmmssfff");
          using (FileStream input = new FileStream(System.Web.Configuration.WebConfigurationManager.AppSettings["filepath"] +
InputFileName, FileMode.Open))
          {
              using (FileStream output = new FileStream(System.Web.Configuration.WebConfigurationManager.AppSettings["filepath"] +
FileName, FileMode.Create))
              {
                  SevenZip.Sdk.Compression.Lzma.Decoder decoder = new
                  SevenZip.Sdk.Compression.Lzma.Decoder(); byte[] properties = new byte[5]; if
                  (input.Read(properties, 0, 5) != 5) throw (new Exception("input .lzma is too
                  short")); decoder.SetDecoderProperties(properties);
```

```
                long outSize = 0; for
                (int i = 0; i < 8; i++)
                { int v =
                    input.ReadByte();
                    if (v < 0)
                        throw (new Exception("Can't
                    Read 1")); outSize |=
                    ((long)(byte)v) << (8 * i);
                }
                long compressedSize = input.Length - input.Position;

                decoder.Code(input, output, compressedSize, outSize, null);
            }
        }
        return FileName;
    }
}
```

12. eSignGateway UI changes applicable only for Aadhaar v2(eMudhra)

eSignGatewayCustomUI

Purpose: To set the custom UI(custUI) for eSignGateway page

DataType: String

Parameter: Optional

Default: Default UI for V2(eMudhra Aadhaar)

Value: Encoded base64 of **Sample**

JSON Format:

```
{
 "userAadhaarLast4Digit": "",
 "logoURL": "",
 "buttonColour": "",
 "headerColour": ""
}
```

Sample Param:

custUI=eyJidXR0b25Db2xvdXIiOiAiRkYwMDAwIiwiaGVhZGVyQ29sb3VyIjogIkZGMDAwMCIsImxvZ29VUkwiOiAiaHR0cHM6Ly9xYXNlcnZlci1pbnQuZW11ZGhyYS5uZXQ6MTgwMDQvZVNpZ25BU1BEZW1vVjIvaW1hZ2VzL2VtdWRocmEtbG9nby5wbmcifQ

12.1 Specifications

Sl No	Parameter	Data Type	Presence	Description
1	buttonColour	String	Optional	Length: 6/7 Value: A valid colour code in RGB hex format ex. #FF0000/FF0000 Default: eSign default button colour
2	headerColour	String	Optional	Length: 6/7 Value: A valid colour code in RGB hex format ex. #FF0000/FF0000 Default: eSign default header colour

3	logoURL	String	Optional	Length: max 100 Recommended size: 165x45 px Format: jpg/png/svg Value: URL of logo Data Type: String (URL) Default: ASP Name in text format
4	userAadhaarLast4Digit	String	Optional	The last 4 digits get validated against the Aadhaar number entered in authenticator page.

12.2 Custom signature appearance changes applicable for Aadhaar v2(eMudhra)

Sl No	Parameter	Data Type	Presence	Description
1	eMudhraV2SignatureType	String	Optional	**Default:** Default signature appearance for V2 Aadhaar(eMudhra) **Value**: 1 (Standard - Standard signature appearance) 2 (Aadhaar - Aadhaar image as a background appearance in signature 3 (Advance) - Custom image as a background appearance in signature
2	eMudhraV2SignatureImage	String	Conditional Mandatory	Only Applicable if eMudhraV2SignatureType=3 **Value**: Base64 Image

3	eMudhraV2SignatureImageExtension	String	Conditional Mandatory	Only Applicable if eMudhraV2SignatureType=3 Value: png/jpg/jpeg (Extension of the file which has been shared in eMudhraV2SignatureImage)